GUÍA DE ESTUDIO

NIVEL: DE CERO A SENIOR



Guía de estudio:

Git desde cero hasta flujos de equipo

Del contenido de .git a los pull requests, la CI y el trabajo con agentes de IA, para quien ya programa y quiere usar Git con criterio senior.

9	16	52	J · M · S
CAPÍTULOS	BLOQUES DE CÓDIGO	FUENTES CITADAS	CHECKLISTS POR NIVEL

NOVIEMBRE 2026

ÍNDICE

Contenido

— **Introducción: Git desde cero hasta flujos de equipo** 3
Cómo leer esta guía 3
Etiquetas de fiabilidad 3
Mapa de la ruta de aprendizaje 3

1 **El modelo interno: objetos, refs e index** 4
1.1 Un almacén clave-valor direccionado por contenido 4
1.2 Blob, tree, commit y tag anotado 4
1.3 Refs y HEAD: una rama es un archivo 5
1.4 El index y las tres áreas 5

2 **Trabajo local diario: commits, ramas, integrar y deshacer** 7
2.1 Un buen commit 7
2.2 Merge: fast-forward, tres vías y conflictos 7
2.3 Rebase 8
2.4 Deshacer con seguridad 8
2.5 Depurar con el historial 8

3 **Remotos y colaboración: fetch, pull, push y pull requests** 10
3.1 Un remoto por dentro y los tres punteros 10
3.2 Push rechazado y force seguro 10
3.3 Pull requests, botones de merge y protección 10
3.4 Secretos 11

4 **Flujos de equipo: ramas, convenciones, automatización y releases** 13
4.1 Estrategias de ramas 13
4.2 Convenciones en cadena 13
4.3 Hooks y pre-commit 14
4.4 CI en el pull request 14
4.5 Releases, hotfixes y merges malos 15

5 **Git hoy y el trabajo con agentes de IA** 16
5.1 switch y restore 16
5.2 Worktrees 16
5.3 Git como red de seguridad con agentes 16

6 **Plan de estudio semanal** 18
Proyecto integrador: una librería Python gestionada como en un equipo 18

7 **Recursos priorizados** 18

8 **Glosario** 19

9 **Respuestas de autoevaluación** 19

A **Apéndice: Índice de fuentes citadas** 20

INTRODUCCIÓN

Git desde cero hasta flujos de equipo

Resumen. Esta guía recorre Git en el orden en que conviene estudiarlo: qué guarda dentro de `.git` (objetos, refs, index), el trabajo local diario (commits, merge, rebase, deshacer, depurar con el historial), los remotos y el pull request, los flujos de equipo (ramas, convenciones, hooks, CI, releases) y, por último, Git reciente y su uso con agentes de IA. Al terminar podrás explicar cada comando por lo que mueve por dentro y recuperar trabajo perdido.

Cómo leer esta guía

Cada capítulo explica el mecanismo antes que la receta y termina con un checklist y preguntas de entrevista. Las salidas de consola marcadas "Git 2.55" se obtuvieron en ejecuciones de prueba hechas para esta guía; los hashes de blobs y trees se reproducen en tu máquina, los de commits no (dependen de autor y fecha). Las preguntas de entrevista son propuestas de la guía, no preguntas recogidas de entrevistas reales. Teclea los ejemplos en un repositorio de prueba.

Etiquetas de fiabilidad

Etiqueta	Significado
Enlace con número volado	Dato leído en la fuente citada durante la investigación.
ref. canónica	Concepto estable de una obra de referencia, no releída ahora.
CG	Conocimiento general del campo, sin fuente concreta.
no verificado	No se pudo confirmar; tómalo como pista, no como hecho.
síntesis	Conclusión propia de la guía a partir de varias fuentes.

Mapa de la ruta de aprendizaje

Qué aporta cada capítulo y qué necesitas antes.

Cap.	Qué aprendes	Antes necesitas
1	Objetos, refs, HEAD, index	Terminal y Python básico
2	Commits, merge, rebase, deshacer, bisect	Cap. 1
3	fetch, pull, push, pull requests, secretos	Caps. 1 y 2
4	Estrategias de ramas, convenciones, hooks, CI, releases	Caps. 2 y 3
5	switch/restore, worktrees, agentes de IA	Caps. 1 a 4
6 a 9	Plan de tres semanas, recursos, glosario, autoevaluación	Todo lo anterior

CAPÍTULO 1

El modelo interno: objetos, refs e index

Los comandos que más confunden (`reset`, `rebase`, el detached HEAD) dejan de hacerlo cuando sabes qué archivos tocan dentro de `.git`. Al terminar podrás reproducir un hash en Python, leer un commit con plumbing y decir qué es una rama.

1.1 Un almacén clave-valor direccionado por contenido

Git es un almacén clave-valor: le das bytes y te devuelve como clave el hash de esos bytes[1]. Todos los objetos se guardan igual: una cabecera con el tipo, un espacio, el tamaño en bytes y un byte nulo; después el contenido. El SHA-1 de ese conjunto es el identificador; el conjunto se comprime con zlib y se escribe en `.git/objects/`, en un subdirectorio con los 2 primeros caracteres del hash y un archivo con los 38 restantes[1]. En Python:

PYTHON

```python
import hashlib


def git_hash(data: bytes, tipo: str = "blob") -> str:
    # Git no hashea el contenido a secas: antepone "<tipo> <bytes>\0"
    cabecera = f"{tipo} {len(data)}\0".encode()
    return hashlib.sha1(cabecera + data).hexdigest()


print(git_hash(b"hola git\n"))
# 358f6e1f281ccf21fdf564dbb0d61b6887371e66  == git hash-object hola.txt
```

1.2 Blob, tree, commit y tag anotado

Hay cuatro tipos de objeto. El **blob** es el contenido de un archivo. El **tree** es un directorio: una lista de entradas con modo, nombre y hash de un blob o de otro tree. El **commit** apunta a un tree raíz y a sus commits padre, y lleva autor, committer y mensaje[1]. El **tag anotado** es un objeto propio con autor, fecha y mensaje que apunta a otro objeto[2]. Con `git cat-file -p` se leen (Git 2.55):

TERMINAL

```
$ git cat-file -p HEAD
tree 4d91873c85bc08ed428e12e38071f4dd44fef08c
author Ana Dev <ana@example.com> 1794387600 +0100
committer Ana Dev <ana@example.com> 1794387600 +0100

Primer commit
```

El commit no contiene un diff: apunta a un tree completo. Git trata sus datos como una serie de instantáneas, y un archivo que no cambió se reutiliza por su hash en vez de guardarse otra vez[3]. Los objetos nacen sueltos, uno por archivo; al ejecutar `git gc` o al hacer push, Git los agrupa en un packfile, donde las ver-

siones parecidas se guardan como deltas y la más reciente queda entera[4]. Como el hash de un commit cubre su tree, sus padres, el autor y las fechas, cambiar cualquiera de ellos produce otro objeto, y el cambio se propaga a todos los descendientes: por eso `amend` y `rebase` generan hashes nuevos CG.

1.3 Refs y HEAD: una rama es un archivo

Una ref es un nombre legible para un hash; las refs viven en `.git/refs`[2]. Con el formato de refs por defecto, una rama es un archivo de texto con los 40 caracteres del hash más un salto de línea: crearla es escribir 41 bytes[5].

TERMINAL

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
8531cefa4eb12abba2925a77081ec7707ae2ab31
```

`HEAD` suele ser una ref simbólica: no apunta a un commit sino a otra ref, la rama actual. En detached HEAD contiene un hash directamente[2]. Un commit hecho así no mueve ninguna rama, y al cambiar de rama nada lo referencia síntesis.

> **Cuidado.** El archivo de la rama puede no existir: `git gc` empaqueta las refs en `.git/packed-refs`[6]. La forma fiable de leer una ref es `git rev-parse <nombre>` o `git show-ref`.

1.4 El index y las tres áreas

Hay tres sitios: el **working tree** (los archivos que editas), el **index** o staging area (el archivo `.git/index`, la propuesta del próximo commit) y el **repositorio** (la base de objetos)[3]. El index no guarda diffs: es una lista plana y ordenada de rutas[7].

`git add` no "marca" el archivo: escribe el blob en `.git/objects` y anota su hash en el index; `git commit` convierte el index en trees, escribe el commit y mueve la rama[1]. El mapa de qué comando copia entre áreas:

```
working tree ──git add──────────────→ index ──git commit──→ repositorio
working tree ←──git restore <ruta>─── index
                                      index ←──git restore --staged── HEAD

git diff            working tree frente a index
git diff --staged   index frente a HEAD
git diff HEAD       working tree frente a HEAD
```

Las tres comparaciones están definidas así en el manual[8].

CHECKLIST POR NIVEL

Checklist de la sección

Nivel	Deberías poder
Junior	Nombrar los cuatro tipos de objeto; explicar las tres áreas.
Mid	Reproducir el hash de un blob en Python; explicar qué escriben `git add` y `git commit`.
Senior	Explicar por qué reescribir historia cambia hashes en cadena; leer una ref sin depender de su archivo.

PREPARACIÓN DE ENTREVISTA

Preguntas de entrevista

Pregunta	Qué incluye una respuesta senior
¿Qué es exactamente una rama?	Una ref con el hash de un commit (archivo o línea en packed-refs); HEAD apunta a ella; crearla no copia nada.
¿Git guarda diffs o instantáneas?	Instantáneas en el modelo (trees completos); deltas solo dentro de packfiles, como compresión.
¿Qué pasa por dentro en `git add` y `git commit`?	add: blob más entrada del index; commit: trees, objeto commit y mover la rama.

CAPÍTULO 2

2

Trabajo local diario: commits, ramas, integrar y deshacer

Aquí se decide la calidad del historial que revisan otros y que usarás para depurar. Al terminar sabrás preparar commits limpios, resolver un conflicto, elegir entre merge y rebase, deshacer sin perder trabajo y usar `bisect`.

2.1 Un buen commit

Un buen commit es un cambio lógico único. El mensaje empieza con una línea de unos 50 caracteres en imperativo, sigue una línea en blanco y después un cuerpo que explica el motivo[9]. La línea en blanco importa: Git trata el texto anterior a ella como título del commit[10]. El ciclo es: editar, `git diff`, `git add -p` (preparar trozo a trozo), `git diff --staged` y `git commit`.

Si olvidaste algo, `git add` y `git commit --amend --no-edit` lo añaden al último commit. `--amend` no edita el commit, crea otro y mueve la rama a él[10]. Por eso solo es seguro si el commit no se ha publicado.

2.2 Merge: fast-forward, tres vías y conflictos

Si la rama actual es ancestro de la que integras, Git solo avanza el puntero (fast-forward) y no crea commit. Si las historias divergieron, usa las dos puntas y su ancestro común, y crea un merge commit con dos padres[11]. Cuando ambos lados cambiaron la misma zona, Git se detiene (Git 2.55):

TERMINAL

```
$ git merge feature/iva-reducido
CONFLICT (content): Merge conflict in precios.py
Automatic merge failed; fix conflicts and then commit the result.
$ tail -n 5 precios.py
<<<<<<< HEAD
IVA = 0.21  # tipo general (ES)
=======
IVA = 0.10  # tipo reducido
>>>>>>> feature/iva-reducido
$ git ls-files -u
100644 2bd2b6ff5a882d849e238bc389dab502e787aab8 1    precios.py
100644 d1964d37f8e9048e0c7a9e5ddfaecd707d72e40a 2    precios.py
100644 e39a7ed5ba176b09cf75c8031eed2cba62d73682 3    precios.py
```

El conflicto vive en el archivo, con marcadores, y en el index, con tres entradas para la misma ruta: stage 1 el ancestro, stage 2 `HEAD`, stage 3 la otra rama[12]. Resolver es editar el archivo, `git add` (que deja una sola entrada de stage 0) y `git commit`; abandonar es `git merge --abort`.

2.3 Rebase

Rebase toma los commits de tu rama que no están en la base nueva y los reaplica uno a uno encima de ella. La instantánea final es la misma que con merge; cambia la forma de la historia[13]. Los commits reaplicados conservan el mensaje pero tienen hashes nuevos: cada uno tiene otro padre (sección 1.2). De ahí la regla: no hagas rebase de commits que ya existen fuera de tu repositorio y sobre los que otra persona pudo basar trabajo[13].

El modo interactivo (`git rebase -i`) abre la lista de commits, el más antiguo arriba, con acciones como `squash`, `fixup` y `drop`[14]. Para arreglar un commit antiguo no publicado: `git commit --fixup=<commit>` y `git rebase -i --autosquash main`, que coloca el arreglo detrás de su destino[15].

2.4 Deshacer con seguridad

`restore` recupera archivos y no mueve la rama; `reset` mueve la punta de la rama; `revert` crea un commit nuevo que invierte otro[16]. `reset` hace hasta tres pasos: mover la rama (`--soft` para ahí), igualar el index a HEAD (`--mixed`, por defecto) e igualar el working tree (`--hard`). Solo `--hard` destruye datos, porque sobrescribe archivos sin confirmar[7]. La tabla de decisión síntesis:

Quiero…	Comando	¿Recuperable?
Descartar cambios sin preparar	`git restore <archivo>`	No
Deshacer el último commit, conservar cambios	`git reset HEAD~1`	Sí
Tirar commits y cambios locales	`git reset --hard <commit>`	Commits sí; lo no confirmado no
Deshacer un commit publicado	`git revert <commit>`	Sí

La red de seguridad es el reflog, que registra cada movimiento de HEAD y de las ramas en el repositorio local[17] (Git 2.55):

TERMINAL

```
$ git reset --hard HEAD~2
HEAD is now at ad68116 Add precio_final
$ git reflog -2
ad68116 HEAD@{0}: reset: moving to HEAD~2
012a15f HEAD@{1}: commit: Add cupon tests
$ git reset --hard 'HEAD@{1}'
HEAD is now at 012a15f Add cupon tests
```

Límites: el reflog es local, caduca y no cubre lo que nunca fue commit.

2.5 Depurar con el historial

Para "¿qué commit rompió esto?", `git bisect` hace una búsqueda binaria entre un commit bueno y uno malo. `git bisect run` la automatiza con el código de salida de un script: 0 es bueno, de 1 a 127 es malo, salvo 125, que significa "no se puede probar, sáltalo"[18].

PYTHON

```python
# check.py: fuera del repositorio, para que no cambie al saltar de commit
import os
import sys

sys.path.insert(0, os.getcwd())  # importar desde el repo, no desde el script
try:
    from precios import precio_final
except Exception:
    sys.exit(125)  # este commit no se puede probar: saltarlo
sys.exit(0 if precio_final(100, 10) == 90 else 1)
```

TERMINAL

```
$ git bisect start HEAD main          # primero el malo, después el bueno
Bisecting: 3 revisions left to test after this (roughly 2 steps)
$ git bisect run python ../check.py
05416734afb414e371a18ac760d75936b052e7d5 is the first 'bad' commit
$ git bisect reset
```

Un script que falla por un motivo ajeno al bug marca "bad" en todo y culpa al commit equivocado; para eso está el 125.

CHECKLIST POR NIVEL

Checklist de la sección

Nivel	Deberías poder
Junior	Revisar con `git diff --staged` antes de cada commit; elegir entre `restore`, `reset` y `revert`.
Mid	Explicar los stages 1, 2 y 3 del index; recuperar commits con el reflog.
Senior	Argumentar merge frente a rebase; automatizar `bisect run`; decir qué no es recuperable.

PREPARACIÓN DE ENTREVISTA

Preguntas de entrevista

Pregunta	Qué incluye una respuesta senior
¿Merge o rebase?	Mismo contenido final, distinta historia; rebase reescribe hashes; regla de lo publicado.
Hiciste `reset --hard` y perdiste commits. ¿Y ahora?	`git reflog` y reset al hash; límites: local, caduca, no cubre lo no confirmado.
¿Cómo encuentras el commit que introdujo una regresión?	`git bisect` entre bueno y malo; `bisect run` y el código 125; script fuera del árbol.

CAPÍTULO 3

3

Remotos y colaboración: fetch, pull, push y pull requests

Al terminar sabrás qué mueve cada comando de red, por qué se rechaza un push, cómo forzar sin pisar trabajo ajeno y qué hacer si un secreto llega al remoto.

3.1 Un remoto por dentro y los tres punteros

Un remoto es un nombre asociado a una URL y a una regla de mapeo (refspec) en `.git/config`. `origin` es solo el nombre que pone `git clone`[19].

Hay entonces tres punteros llamados "main" síntesis: tu rama `main`; `origin/main`, una ref local de solo lectura que recuerda dónde estaba la rama del servidor la última vez que te conectaste[19]; y la rama real en el servidor.

- `git fetch` descarga objetos y mueve `origin/*`; no toca tu rama ni tus archivos[20].
- `git pull` ejecuta `fetch` y después integra en la rama actual: solo fast-forward (`--ff-only`, el comportamiento por defecto si no indicas otro), `--rebase` o merge con `--no-rebase`[21].

3.2 Push rechazado y force seguro

El servidor solo acepta mover una rama de A a B si B desciende de A. Si alguien empujó antes, tu commit no desciende del suyo y aceptarlo descartaría su trabajo: `! [rejected] main -> main (non-fast-forward)`[22,23]. La salida normal es integrar y reintentar: `git pull --rebase` y `git push`.

`--force` desactiva la comprobación y puede hacer que el remoto pierda commits. `--force-with-lease` solo sobrescribe si la rama del servidor sigue donde dice tu rama de seguimiento remoto[22].

BASH

```
git fetch origin
git rebase origin/main
git push --force-with-lease --force-if-includes
```

Cuidado. `--force-with-lease` deja de proteger si algo hace `git fetch` en segundo plano (un IDE, un asistente): tu `origin/rama` se actualiza y el lease coincide con commits que nunca viste. `--force-if-includes` añade la comprobación de que integraste en local la punta remota[22].

3.3 Pull requests, botones de merge y protección

Un pull request es una propuesta de fusionar cambios en un proyecto, con conversación, diff y checks; es una función de la plataforma, no de Git[24].

Los tres botones dejan historias distintas[25]:

```
Antes:    main:    A──B
          feature:      └──C──D

Create a merge commit:   A──B──────────M     M tiene dos padres (B y D)
                             └──C──D──┘

Squash and merge:        A──B──S            S = C + D en un commit nuevo

Rebase and merge:        A──B──C'──D'       mismos cambios, hashes nuevos
```

Una rama protegida bloquea por defecto el force push y el borrado, y puede exigir revisiones, status checks, conversaciones resueltas, commits firmados o historia lineal[26].

3.4 Secretos

`.gitignore` solo afecta a archivos sin seguimiento; uno ya rastreado se sigue subiendo[27].

BASH

```bash
git rm --cached .env          # lo saca del index, no lo borra del disco
echo '.env' >> .gitignore
git commit -m "Stop tracking .env"
```

Importante. Esto no borra el secreto de los commits anteriores. Si una credencial llegó al remoto, lo primero es revocarla o rotarla; reescribir la historia con `git-filter-repo` viene después y no alcanza a clones ni forks[28]. La prevención barata es revisar `git diff --staged` y activar push protection, que bloquea los pushes con secretos antes de que entren[29].

CHECKLIST POR NIVEL

Checklist de la sección

Nivel	Deberías poder
Junior	Explicar qué es `origin/main`; distinguir `fetch` de `pull`.
Mid	Resolver un push rechazado sin forzar; elegir botón de merge sabiendo qué historia deja.
Senior	Explicar cuándo falla `--force-with-lease`; responder a un secreto filtrado, empezando por rotarlo.

PREPARACIÓN DE ENTREVISTA

Preguntas de entrevista

Pregunta	Qué incluye una respuesta senior
¿`--force` o `--force-with-lease`?	Compare-and-swap contra la rama de seguimiento; fallo con fetch en segundo plano; `--force-if-includes`.
¿Merge commit, squash o rebase al fusionar un PR?	Historia que deja cada uno; efecto en los hashes.
Subiste una API key. ¿Qué haces?	Rotar primero; después filter-repo y sus costes; prevención con push protection.

CAPÍTULO 4

4

Flujos de equipo: ramas, convenciones, automatización y releases

Un equipo acuerda cuánto vive una rama, cómo se nombran los cambios y qué se comprueba antes de fusionar. Al terminar podrás elegir una estrategia de ramas, montar hooks y CI y gestionar un hotfix o un merge malo en `main`.

4.1 Estrategias de ramas

Las cuatro estrategias conocidas difieren en una variable: cuánto tiempo vive el código separado de la rama principal y cuántas ramas largas hay síntesis.

Estrategia	Vida de la rama de trabajo	Encaja cuando
Git Flow	Días o semanas	Varias versiones mantenidas a la vez
GitHub Flow	Lo que dure el PR	Servicio web con una versión en producción
GitLab Flow	Corta	El despliegue no es automático en cada merge
Trunk-based	Horas, como mucho un par de días	CI rápida y tests fiables

El autor de Git Flow escribió en 2020 que para aplicaciones web con entrega continua recomienda un flujo más simple, como GitHub Flow, y que su modelo sigue encajando en software con versiones explícitas[30].

La evidencia: el informe State of DevOps 2016 asocia con mejor rendimiento de entrega las ramas que viven menos de un día, tener "less than three active branches in total", fusionar en el tronco a diario y no tener periodos de congelación[31].

4.2 Convenciones en cadena

Tres convenciones se encadenan síntesis. El mensaje declara el tipo de cambio con el formato `<type>[optional scope]: <description>`; `fix` corrige un bug, `feat` añade funcionalidad y un `!` tras el tipo o un pie `BREAKING CHANGE:` marca un cambio incompatible[32]. El tipo decide el número MAJOR.MINOR.PATCH: `fix` sube PATCH, `feat` sube MINOR y el cambio incompatible sube MAJOR; una versión publicada no se modifica[33]. La versión se fija con un tag anotado, que guarda autor, fecha y mensaje; `git describe` ignora por defecto los tags ligeros[34].

BASH

```
git tag -a v1.4.0 -m "Release 1.4.0"   # objeto tag: autor, fecha, mensaje
git push --follow-tags                 # un push normal no sube los tags
```

4.3 Hooks y pre-commit

Un hook es un ejecutable con nombre fijo en `.git/hooks/` que Git lanza en un momento concreto. `commit-msg` recibe como único parámetro la ruta del archivo con el mensaje; si sale con código distinto de cero, el commit se cancela[35]. Escrito a mano se ve el mecanismo:

PYTHON

```python
#!/usr/bin/env python3
"""Rechaza el commit si la primera línea no sigue Conventional Commits."""
import re
import sys

PATRON = re.compile(r"^(feat|fix|docs|refactor|perf|test|build|ci|chore)"
                    r"(\([\w\-./]+\))?!?: .+")

with open(sys.argv[1], encoding="utf-8") as f:  # Git pasa la ruta del mensaje
    primera = f.readline().strip()

if not primera.startswith("Merge ") and not PATRON.match(primera):
    print(f"Mensaje no válido: {primera!r}", file=sys.stderr)
    sys.exit(1)  # distinto de cero: Git cancela el commit
```

Este hook no protege nada. Los hooks de cliente no se copian al clonar[36] y se saltan con `--no-verify`[35]. La regla se impone en el servidor (síntesis). El framework `pre-commit` resuelve el reparto: un archivo versionado, `.pre-commit-config.yaml`, declara los hooks y su versión, y `pre-commit install` escribe el script en `.git/hooks`[37].

4.4 CI en el pull request

La misma comprobación va en dos sitios: en local por rapidez y en CI por garantía (síntesis). El workflow siguiente es un ejemplo montado para esta guía con piezas de la documentación oficial; no se ejecutó en un repositorio real.

YAML

```yaml
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
    branches: [main]
permissions:
  contents: read
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
      - run: pip install -r requirements.txt pytest pre-commit
      - run: pre-commit run --all-files
      - run: pytest
```

`permissions` limita el token a lectura y `concurrency` cancela ejecuciones antiguas de la misma rama[38]; las versiones de las acciones son las de la guía oficial[39]. En el evento `pull_request`, `GITHUB_SHA` es el commit de merge del PR con la base, no la punta de tu rama[40]: la CI puede fallar aunque tus tests pasen en local, porque `main` avanzó.

4.5 Releases, hotfixes y merges malos

Se publica desde el tronco, etiquetando un commit de `main`, o desde una rama de release cortada poco antes, que solo recibe arreglos. En ambos casos el bug se arregla primero en `main` y se lleva a la release con cherry-pick; al revés, el arreglo se olvida y reaparece en la versión siguiente[41].

BASH

```bash
git switch -c release-1.5 v1.5.0     # rama creada a posteriori desde el tag
git cherry-pick -x abc1234           # el arreglo, ya fusionado en main
git tag -a v1.5.1 -m "Release 1.5.1"

git revert -m 1 9f8e7d6              # deshace en main el merge 9f8e7d6
```

`-x` anota en el mensaje el commit de origen[42]. La última línea es otro caso: un merge malo en `main` no se borra, se revierte con un commit nuevo; `-m 1` indica qué padre es la línea principal[43].

> **Cuidado.** Tras revertir un merge, volver a fusionar esa rama no trae sus cambios originales: sus commits ya son ancestros de `main`. Hay que revertir antes el revert[44].

CHECKLIST POR NIVEL

Checklist de la sección

Nivel	Deberías poder
Junior	Escribir mensajes Conventional Commits; instalar `pre-commit`.
Mid	Relacionar `fix`, `feat` y `!` con SemVer; publicar un tag anotado; escribir un workflow de CI.
Senior	Elegir estrategia de ramas según versiones en producción; explicar por qué un hook no es una frontera de seguridad; revertir un merge y recuperarlo después.

PREPARACIÓN DE ENTREVISTA

Preguntas de entrevista

Pregunta	Qué incluye una respuesta senior
¿Git Flow, GitHub Flow o trunk-based para un servicio web?	Vida de la rama y versiones vivas; nota de 2020 de Driessen; criterios de DORA.
¿Cómo garantizas que nadie sube código sin formatear?	Hooks locales por rapidez; garantía en CI obligatoria; `--no-verify` y que los hooks no se clonan.
¿Cómo deshaces un merge malo en `main`?	`git revert -m 1`; qué es el padre 1; revert del revert al volver a fusionar.

CAPÍTULO 5

Git hoy y el trabajo con agentes de IA

Al terminar sabrás qué comandos usar hoy, cómo abrir varias ramas a la vez con worktrees y qué límites poner a un agente.

5.1 switch y restore

Git 2.23 repartió el trabajo de `git checkout` en dos comandos: `git switch` cambia de rama y `git restore` recupera archivos[45]. `checkout` no se elimina[46].

5.2 Worktrees

Un worktree es un segundo directorio de trabajo unido al mismo repositorio: comparte objetos y refs, pero tiene su propio `HEAD` y su propio index. Una rama solo puede estar abierta en un worktree a la vez[47].

BASH

```
git worktree add ../proyecto-feature-a -b feature-a   # rama nueva, otro directorio
git worktree list
git worktree remove ../proyecto-feature-a             # exige worktree limpio
```

Dos límites síntesis: el worktree aísla archivos, no puertos ni la base de datos local; y no evita los conflictos, los pospone hasta integrar las ramas.

5.3 Git como red de seguridad con agentes

Los checkpoints de Claude Code sirven para recuperar dentro de la sesión, pero no rastrean los archivos que cambió un comando de Bash ni las ediciones de la mayoría de subagentes; para eso la documentación remite a Git[48]. De ahí la rutina síntesis: haz un commit antes de cada tarea del agente, trabaja siempre en una rama y revisa lo que generó con `git status`, `git add -p` leyendo cada trozo y `git diff --staged`.

Las reglas de permisos las aplica la herramienta, no el modelo[49].

> **Advertencia.** Las reglas por patrón de texto son frágiles: `Bash(git push *)` no coincide con `git -C . push origin main` [49]. La frontera que no depende del agente está en el servidor: rama protegida, CI obligatoria y push protection (capítulos 3 y 4) síntesis.

CHECKLIST POR NIVEL

Checklist de la sección

Nivel	Deberías poder
Junior	Usar `switch` y `restore`; revisar con `git add -p` lo que generó un asistente.
Mid	Crear, listar y eliminar worktrees; explicar qué comparten y qué no.
Senior	Poner límites a un agente en herramienta y servidor; explicar por qué un checkpoint no sustituye a un commit.

PREPARACIÓN DE ENTREVISTA

Preguntas de entrevista

Pregunta	Qué incluye una respuesta senior
¿Cómo lanzas varios agentes de IA en un mismo repositorio?	Un worktree y una rama por agente; qué se comparte; no aísla puertos ni bases de datos.
¿Por qué no basta con decirle al agente "no hagas push"?	Permisos de la herramienta; fragilidad de los patrones; protección en el servidor.

CAPÍTULO 6

Plan de estudio semanal

Tres semanas, a razón de una hora diaria.

Semana	Capítulos	Práctica	Prueba de terminado
1	1 y 2	Reproducir hashes; provocar un conflicto; recuperar commits; `bisect run`	Recuperas un `reset --hard`
2	3	Dos clones; push rechazado; PR con cada botón de merge; proteger `main`	Resuelves un rechazo sin `--force`
3	4 y 5	`pre-commit`, CI, tag; hotfix con cherry-pick; un agente en un worktree	La CI bloquea un PR con errores

Proyecto integrador: una librería Python gestionada como en un equipo

Una librería pequeña (por ejemplo, el módulo `precios`) en GitHub, por etapas.

1. **Inspección.** Un script calcula el hash de un blob. Hecho cuando: coincide con `git hash-object`.
2. **Historial.** Limpia una rama con `--autosquash` y busca un bug con `bisect run`. Hecho cuando: bisect señala el commit correcto.
3. **Colaboración.** Protege `main` y fusiona un PR. Hecho cuando: un push directo a `main` es rechazado.
4. **Automatización.** Añade `pre-commit` y CI. Hecho cuando: un PR con un error de lint no se puede fusionar.
5. **Hotfix.** Lleva un arreglo con `cherry-pick -x` a `release-0.1`. Hecho cuando: `git describe` devuelve allí `v0.1.1`.
6. **Agente.** Encarga una tarea en un worktree. Hecho cuando: puedes explicar el PR línea a línea.

CAPÍTULO 7

Recursos priorizados

Gratuitos y en inglés, en orden de uso.

Prioridad	Recurso	Para qué	Cuándo
1	Pro Git[50]	Libro de referencia: ramas, herramientas e internals	Semanas 1 y 2
2	DORA, Trunk-based development[51]	Evidencia sobre ramas cortas	Semana 3
3	pre-commit[37]	Montar hooks compartidos	Etapa 4 del proyecto
4	Claude Code, Worktrees[52]	Agentes en paralelo y sus límites	Etapa 6 del proyecto

CAPÍTULO 8

Glosario

Término	Definición
blob	Objeto que guarda el contenido de un archivo, sin nombre.
detached HEAD	Estado en que `HEAD` contiene un hash en lugar de apuntar a una rama.
fast-forward	Integración que solo avanza el puntero de la rama, sin crear commit.
index	Archivo `.git/index`: la propuesta del próximo commit (staging area).
ref	Nombre que apunta a un hash: ramas, tags, ramas de seguimiento remoto.
reflog	Registro local de los movimientos de `HEAD` y de las ramas.
refspec	Regla que mapea refs del remoto a refs locales.
tree	Objeto que representa un directorio: modos, nombres y hashes.
upstream	Rama remota que sigue una rama local.
worktree	Directorio de trabajo adicional unido al mismo repositorio.

CAPÍTULO 9

Respuestas de autoevaluación

Responde sin mirar y compara.

1. **¿Por qué dos archivos con el mismo contenido y distinto nombre ocupan un solo blob?** Porque el hash se calcula sobre la cabecera `blob <tamaño>\0` y el contenido; el nombre vive en el tree, no en el blob.
2. **Tras un rebase de tu rama ya publicada, `git push` falla. ¿Por qué y qué haces?** Los commits tienen hashes nuevos y no descienden de la rama remota. Si la rama es solo tuya, `git push --force-with-lease --force-if-includes`; si no, integra con merge.
3. **¿Qué diferencia hay entre revertir un commit normal y revertir un merge?** El merge tiene dos padres y hay que indicar la línea principal con `-m 1`. Además, volver a fusionar la misma rama exige revertir antes el revert.
4. **Un agente ejecutó `git reset --hard` en tu rama. ¿Qué se recupera?** Lo que estaba en commits, con `git reflog`. Lo que nunca se confirmó ni se guardó en un stash se ha perdido; los checkpoints del asistente no cubren los cambios hechos por comandos de shell.

APÉNDICE

Índice de fuentes citadas

Cada número volado del texto (p. ej. [12]) remite a esta lista. Las URL idénticas se agrupan en una sola entrada; si una misma fuente aparece con títulos distintos en el texto, se muestran todos. Entre paréntesis, el capítulo donde se cita. Todas las direcciones son enlaces activos. Total: 52 fuentes únicas.

1 **Pro Git, 10.2 Git Objects** (cap. 1)
https://git-scm.com/book/en/v2/Git-Internals-Git-Objects

2 **Pro Git, 10.3 Git References** (cap. 1)
https://git-scm.com/book/en/v2/Git-Internals-Git-References

3 **Pro Git, 1.3 What is Git?** (cap. 1)
https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F

4 **Pro Git, 10.4 Packfiles** (cap. 1)
https://git-scm.com/book/en/v2/Git-Internals-Packfiles

5 **Pro Git, Branches in a Nutshell** (cap. 1)
https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

6 **Pro Git, 10.7 Maintenance and Data Recovery** (cap. 1)
https://git-scm.com/book/en/v2/Git-Internals-Maintenance-and-Data-Recovery

7 **Pro Git, 7.7 Reset Demystified** (cap. 1, cap. 2)
https://git-scm.com/book/en/v2/Git-Tools-Reset-Demystified

8 **git-diff** (cap. 1)
https://git-scm.com/docs/git-diff

9 **Pro Git, Contributing to a Project** (cap. 2)
https://git-scm.com/book/en/v2/Distributed-Git-Contributing-to-a-Project

10 **git-commit** (cap. 2)
https://git-scm.com/docs/git-commit

11 **Pro Git, Basic Branching and Merging** (cap. 2)
https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging

12 **git-merge** (cap. 2)
https://git-scm.com/docs/git-merge

13 **Pro Git, Rebasing** (cap. 2)
https://git-scm.com/book/en/v2/Git-Branching-Rebasing

14 **Pro Git, Rewriting History** (cap. 2)
https://git-scm.com/book/en/v2/Git-Tools-Rewriting-History

15 **git-rebase** (cap. 2)
https://git-scm.com/docs/git-rebase

16 **git** (cap. 2)
https://git-scm.com/docs/git

17 **git-reflog** (cap. 2)
https://git-scm.com/docs/git-reflog

18 **git-bisect** (cap. 2)
https://git-scm.com/docs/git-bisect

19 **Pro Git, 3.5 Remote Branches** (cap. 3)
https://git-scm.com/book/en/v2/Git-Branching-Remote-Branches

20 **git-fetch** (cap. 3)
https://git-scm.com/docs/git-fetch

21 **git-pull** (cap. 3)
https://git-scm.com/docs/git-pull

22 **git-push** (cap. 3)
https://git-scm.com/docs/git-push

23 **Dealing with non-fast-forward errors** (cap. 3)
https://docs.github.com/en/get-started/using-git/dealing-with-non-fast-forward-errors

24 **About pull requests** (cap. 3)
https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

25 **About pull request merges** (cap. 3)
https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/incorporating-changes-from-a-pull-request/about-pull-request-merges

26 **About protected branches** (cap. 3)
https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

27 **gitignore** (cap. 3)
https://git-scm.com/docs/gitignore

28 **Removing sensitive data from a repository** (cap. 3)
https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository

29 **About push protection** (cap. 3)
https://docs.github.com/en/code-security/secret-scanning/introduction/about-push-protection

30 **Driessen, A successful Git branching model** (cap. 4)
https://nvie.com/posts/a-successful-git-branching-model/

31 **State of DevOps 2016, p. 31** (cap. 4)
https://dora.dev/publications/pdf/state-of-devops-2016.pdf

32 **Conventional Commits 1.0.0** (cap. 4)
https://www.conventionalcommits.org/en/v1.0.0/

33 **Semantic Versioning 2.0.0** (cap. 4)
https://semver.org/

34 **git-tag** (cap. 4)
https://git-scm.com/docs/git-tag

35 **githooks** (cap. 4)
https://git-scm.com/docs/githooks

36 **Pro Git, Git Hooks** (cap. 4)
https://git-scm.com/book/en/v2/Customizing-Git-Git-Hooks

37 **pre-commit** (cap. 4, cap. 7)
https://pre-commit.com/

38 **Workflow syntax** (cap. 4)
https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

39 **Building and testing Python** (cap. 4)
https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-python

40 **Events that trigger workflows** (cap. 4)
https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows

41 **Branch for release** (cap. 4)
https://trunkbaseddevelopment.com/branch-for-release/

42 **git-cherry-pick** (cap. 4)
https://git-scm.com/docs/git-cherry-pick

43 **git-revert** (cap. 4)
https://git-scm.com/docs/git-revert

44 **revert-a-faulty-merge** (cap. 4)
https://github.com/git/git/blob/HEAD/Documentation/howto/revert-a-faulty-merge.adoc

45 **GitHub Blog, Highlights from Git 2.23** (cap. 5)
https://github.blog/open-source/git/highlights-from-git-2-23/

46 **BreakingChanges** (cap. 5)
https://git-scm.com/docs/BreakingChanges

47 **git-worktree** (cap. 5)
https://git-scm.com/docs/git-worktree

48 **Claude Code, Checkpointing** (cap. 5)
https://code.claude.com/docs/en/checkpointing

49 **Claude Code, Permissions** (cap. 5)
https://code.claude.com/docs/en/permissions

50 **Pro Git** (cap. 7)
https://git-scm.com/book/en/v2

51 **DORA, Trunk-based development** (cap. 7)
https://dora.dev/capabilities/trunk-based-development/

52 **Claude Code, Worktrees** (cap. 7)
https://code.claude.com/docs/en/worktrees